

Pake Plus — 项目分工与功能说明

一、项目介绍

项目名称

Pake Plus — 基于 Pake 的网页桌面应用增强工具

项目背景

Pake 是一个开源 CLI 工具（Rust + Tauri v2），能将任意网页打包为轻量级原生桌面应用。原项目提供了基础窗口管理、系统托盘、快捷键、通知等能力，但作为日常使用的桌面应用，仍缺少一些高频刚需功能。

项目目标

在 Pake 的基础上新增四个独立功能模块，使其更接近一个成熟的桌面浏览器应用：

模块	负责成员	一句话描述
广告/跟踪拦截	成员 A	内置广告过滤引擎，自动拦截网页广告和跟踪请求
离线缓存代理	成员 B	HTTP 层请求拦截与磁盘缓存，断网也能访问已浏览页面
剪贴板管理	成员 C	系统级剪贴板监听与历史记录，离线搜索随时复用
设置面板 + 视频	成员 D	可视化设置页面，统一管理所有功能配置

与原项目 Pake 的区别

对比维度	原 Pake	Pake Plus
广告处理	无任何广告过滤能力	内置 EasyList 规则引擎，网络层拦截 + DOM 元素隐藏双重过滤
离线能力	完全依赖网络，断网即不可用	HTTP 层代理缓存，已访问页面断网后仍可浏览
剪贴板能力	依赖系统剪贴板，无历史记录	系统级剪贴板监听，自动记录历史，支持搜索和复用
功能配置	仅支持打包时通过 CLI 参数一次性配置	运行时可视化设置面板，修改即时生效并持久化
配置持久化	配置在打包时写入，运行时不可改	运行时可修改配置，自动写入本地文件，下次启动自动加载

二、成员 A — 广告/跟踪拦截

功能概述

为 Pake 打包的桌面应用增加内置广告过滤能力，让用户获得清爽的网页浏览体验。该功能在网络请求层面和页面 DOM 层面同时工作，形成双重过滤。

功能详情

功能 A1：网络请求拦截

效果描述：当网页尝试加载广告或跟踪脚本时，拦截引擎自动识别并阻止该请求发出。被拦截的请求不会消耗网络流量，页面加载速度明显加快。

用户感知：

- 广告位显示为空白，而非"加载失败"的占位图
- 页面加载速度相比未拦截时提升 30%-50%（取决于广告数量）
- 不再有弹窗广告、视频贴片广告

工作方式：基于 EasyList 规则集（全球最主流的广告过滤规则，也是 Adblock Plus / uBlock Origin 使用的同一套规则），在 Rust 后端对每一个网络请求的 URL 进行匹配判断。匹配到黑名单域名的请求直接拒绝发出，匹配到 URL 正则的请求同样被拦截。

功能 A2：DOM 元素隐藏

效果描述：除了拦截网络请求，CSS 选择器规则会注入到页面中，将广告容器元素设置为不可见。即使某些广告通过内联方式绕过网络拦截，其 DOM 元素仍会被隐藏。

用户感知：

- 页面布局更加紧凑，广告占位区域彻底消失
- 文章中间的穿插广告被移除，阅读体验更流畅
- 侧边栏推荐广告不再显示

工作方式：从 EasyList 规则中提取元素隐藏规则（`##.class`、`###id` 等格式），注入为 CSS 样式。同时监听页面的 DOM 变化，对动态插入的广告元素实时应用隐藏规则。

功能 A3：自定义规则

效果描述：用户可以添加自己的拦截规则，不受 EasyList 内置规则限制。

典型使用场景：

- 拦截某个网站特有的跟踪脚本
- 隐藏某个网站烦人的订阅弹窗
- 屏蔽特定域名的所有请求

规则格式：支持三种规则格式——

- 域名拦截：`||doubleclick.net^`（拦截该域名的所有请求）
- URL 正则：`/advert/banner/`（拦截 URL 中包含该模式的请求）
- 元素隐藏：`###subscribe-modal`（隐藏对应 ID 的元素）、`##.popup-ad`（隐藏对应 class 的元素）

功能 A4：拦截统计

效果描述：托盘图标右下角显示当前页面被拦截的请求数量，用户可直观看到广告拦截的工作状态。

用户感知：

- 托盘图标角标显示数字（如 "12"），表示当前页面已拦截 12 个广告/跟踪请求
- 切换页面后计数重置
- 鼠标悬停托盘图标时显示 "已拦截 XX 个请求" 的提示文字

功能 A5：CLI 选项 --block-ads

效果描述：在打包桌面应用时，通过命令行参数决定是否启用广告拦截功能。

使用示例：

```
1 pake https://example.com --block-ads
```

打包后的应用自带广告拦截，无需用户额外配置。

功能 A6：CLI 选项 --adblock-rules

效果描述：在打包时指定一个自定义规则文件，这些规则会和内置 EasyList 规则合并生效。

使用示例：

```
1 pake https://example.com --block-ads --adblock-rules ./my-rules.txt
```

适合企业场景：为特定业务系统的内嵌网页定制拦截规则。

三、成员 B — 离线缓存代理

功能概述

为 Pake 打包的桌面应用增加 HTTP 层离线缓存能力。Rust 后端在 HTTP 请求层做透明代理，将浏览过的网页、图片、CSS、JS 等资源自动缓存到磁盘。当网络断开时，已缓存的内容仍可直接从本地加载，页面正常显示而非报错。这是一个只有通过系统级网络层拦截才能实现的功能，JS 完全无法做到。

为什么需要 Rust

- **HTTP 层拦截：**在 Tauri 的 `on_navigation` 和资源加载层面拦截所有网络请求，JS 层面的 Service Worker 无法覆盖全部资源类型
- **异步磁盘 I/O：**大量并发请求的缓存写入、索引维护，需要 `tokio` 异步运行时和高效的 I/O 调度
- **LRU 淘汰算法：**磁盘空间有限，需要维护缓存索引和淘汰策略，这是典型的系统编程问题
- **离线服务：**断网时从本地磁盘重建 HTTP 响应，需要理解 HTTP 协议头（Cache-Control、ETag 等）

功能详情

功能 B1：透明 HTTP 代理拦截

效果描述：用户在浏览网页时完全无感知。应用在后台自动拦截每个 HTTP 请求，检查本地是否有缓存副本。有则对比新鲜度决定是否使用缓存，无则正常发起网络请求并在响应后将内容写入缓存。

用户感知：

- 正常上网时，页面加载速度不变甚至更快（命中缓存时直接本地返回，延迟从几百 ms 降到几 ms）
- 第二次打开同一个页面时，图片和静态资源瞬间加载（从磁盘缓存而非网络）

工作方式：利用 Tauri 的资源拦截能力，在 Rust 侧注册 HTTP 拦截器。每个 GET 请求先查本地缓存索引，命中且未过期则直接返回缓存数据；未命中或已过期则放行请求、获取响应后异步写入磁盘缓存。

功能 B2：磁盘缓存管理

效果描述：缓存文件存储在应用数据目录下，按 URL 的哈希值作为文件名，索引文件记录 URL ↔ 缓存文件的映射、缓存时间、文件大小、HTTP 响应头（Content-Type、ETag、Cache-Control）。

缓存策略：

- 缓存内容：HTML 页面、CSS 样式、JS 脚本、图片（PNG/JPG/GIF/SVG/WebP）、字体文件
- 默认缓存大小上限：200MB
- 单个文件上限：10MB（超过不缓存）
- 过期策略：遵循服务器返回的 `Cache-Control: max-age` 值；无此头时默认缓存 24 小时

功能 B3：LRU 淘汰算法

效果描述：当缓存总大小超过上限时，自动删除最久未被使用的缓存文件，释放磁盘空间。

淘汰规则：

- 维护每个缓存文件的"最后访问时间"
- 磁盘空间超过 200MB 上限时，从最老的开始删除
- 每次淘汰 10MB（约占总量 5%），避免频繁触发
- 正在使用的缓存文件不会被淘汰

功能 B4：离线模式

效果描述：当网络断开时，已缓存的页面依然可以正常访问。用户打开之前浏览过的页面，内容从本地缓存加载，页面可以正常浏览（只是无法跳转到未缓存的链接）。

用户感知：

- 断网状态下打开已缓存的页面 → 正常显示，仅地址栏旁出现一个小灰色"离线"标识
- 尝试访问未缓存的新页面 → 显示友好的离线提示页，建议稍后重试
- 点击已缓存页面的内部链接 → 如果目标页也被缓存过，正常跳转；否则提示未缓存

离线提示页效果：一个简洁的页面，显示"当前处于离线状态，该页面未缓存"，并列出生最近缓存的页面供用户直接点击打开。

功能 B5：缓存统计面板

效果描述： 在设置面板的缓存区域，用户可以查看缓存使用情况。

显示信息：

- 已缓存文件总数（如 "1,247 个文件"）
- 磁盘占用（如 "168 MB / 200 MB"）
- 缓存命中率（如 "最近一小时命中率: 73%"）
- "清空缓存"按钮

功能 B6：CLI 选项 --cache

效果描述： 在打包时决定是否为该应用启用离线缓存功能。

```
1 pake https://example.com --cache
```

功能 B7：CLI 选项 --cache-size

效果描述： 在打包时指定缓存大小上限（单位 MB）。

```
1 pake https://example.com --cache --cache-size 500
```

四、成员 C — 剪贴板管理

功能概述

为 Pake 打包的桌面应用增加系统级剪贴板管理能力。自动监听系统剪贴板的每一次文本变化，将复制内容记录下来，形成可搜索的历史列表。用户可以随时打开剪贴板面板，搜索和复用之前复制过的任意内容。这是纯粹的 OS 层功能——JS 在浏览器沙箱中无法监听系统剪贴板，必须通过 Rust FFI 调用操作系统 API 才能实现。

为什么需要 Rust

- **系统级剪贴板监听：** Windows 需要 `AddClipboardFormatListener`，macOS 需要轮询 `NSPasteboard`，Linux 需要监听 X11 `selection` 事件或 Wayland `wl_data_device`。这些都是平台原生 API，必须通过 Rust FFI 调用
- **跨平台 FFI：** 三套完全不同的 OS API，用 Rust 的 `#[cfg(target_os)]` 做条件编译统一接口，是 Rust 相比 C/C++ 的典型优势场景
- **去重与索引：** 剪贴板数据量可能很大，需要高效的哈希去重和全文搜索，CPU 密集操作适合 Rust 处理
- **后台常驻：** 剪贴板监听是持续运行的循环，需要低 CPU 占用的高效轮询，Rust 的无 GC、低内存占用特别适合

功能详情

功能 C1：系统剪贴板自动监听

效果描述：应用启动后，剪贴板监听器在后台静默运行。每次用户 Ctrl+C（或右键复制）一段文字，监听器在几百毫秒内检测到变化，将新内容存入本地数据库。用户无需任何操作，一切自动完成。

用户感知：

- 完全无感知，不会弹窗，不会打断正常操作
- 托盘图标有微小变化提示"正在监听中"（图标右下角一个小圆点）
- 复制后，内容已自动被记录

工作方式：Rust 后端启动一个独立线程，轮询系统剪贴板（Windows 用

`AddClipboardFormatListener` 注册回调，macOS 用

`NSPasteboard.generalPasteboard.changeCount` 比对数，Linux 用 X11 `XFixesSelectionNotify` 事件）。检测到文本变化后，计算哈希值去重，写入本地存储。

功能 C2：剪贴板历史面板

效果描述：按 Ctrl+Shift+V 弹出一个浮层面板，显示最近的剪贴板历史列表。

面板外观：

- 浮层在屏幕右下角弹出，宽约 320px，高约 480px
- 顶部有一个搜索框，输入关键词可即时过滤历史记录
- 每条记录显示：纯文本内容（截取前两行预览）、复制时间（如 "3 分钟前"）、来源应用图标（可选，若有）
- 鼠标悬停某条时，右侧出现"复制"和"删除"按钮
- 面板底部显示总条数统计（如 "共 1,247 条"）和"清空全部"按钮
- 点击面板外任意区域或按 ESC 关闭面板

功能 C3：全文搜索

效果描述：在剪贴板面板顶部的搜索框输入关键词，历史列表即时过滤，只显示包含该关键词的记录。

搜索行为：

- 支持中文/英文模糊匹配（包含即匹配）
- 输入即搜，无需按回车，过滤结果实时更新
- 支持多个关键词空格分隔（AND 逻辑）
- 搜索无结果时显示 "无匹配记录"

工作方式：在 Rust 后端维护一个全文索引。简单的实现方式是每次搜索时遍历所有记录的文本进行字符串包含匹配（数据量不大时性能足够）；如果需要更强性能，可以用 `tantivy` 等 Rust 全文搜索库建立倒排索引。

功能 C4：一键复制复用

效果描述：在剪贴板面板中点击任意一条历史记录，该内容立即写入系统剪贴板。用户可以回到任意应用中 Ctrl+V 粘贴。

交互细节：

- 点击记录本身 → 复制到剪贴板 + 面板自动关闭 + 屏幕底部弹出短 toast "已复制"
- 如果记录是长文本，点击后展开显示全文（避免面板内显示不全）
- 支持连续点击多条，每次都会更新剪贴板

功能 C5：隐私与过滤

效果描述：用户可以设置哪些应用或哪些类型的内容不记录到剪贴板历史中，保护密码、银行卡号等敏感信息。

过滤规则：

- 内容长度过滤：小于 2 个字符不记录（单个字母、标点等无意义复制）
- 内容长度上限：超过 10,000 字符的文本不记录（如复制整个文件内容）
- 敏感信息识别：自动检测并跳过像是密码格式的文本（如 6-30 位字母数字组合、信用卡号格式）

功能 C6：自动清理

效果描述：剪贴板历史不会无限增长，系统自动按规则清理旧数据。

清理策略（满足任一条件即触发）：

- 记录总数超过 2000 条 → 删除最早的 500 条
- 记录时间超过 30 天 → 自动删除
- 用户手动点击"清空全部" → 立即删除所有记录（需二次确认）

功能 C7：快捷键入口

快捷键	功能
Ctrl+Shift+V	打开剪贴板历史面板
ESC	关闭面板
点击记录	复制该条到剪贴板并关闭面板

功能 C8：CLI 选项 --clipboard

效果描述：在打包时决定是否为该应用启用剪贴板管理功能。

```
1 pake https://example.com --clipboard
```

功能 C9：CLI 选项 --clipboard-max

效果描述：在打包时指定剪贴板历史最大记录数。

```
1 pake https://example.com --clipboard --clipboard-max 5000
```

五、成员 D — 设置面板 + 数据迁移 + 诊断 + 视频

功能概述

D 同学负责四个层面：一是可视化的设置面板，图形化配置 A/B/C 模块的所有选项；二是数据导出/导入功能，可打包应用数据一键迁移到另一台电脑；三是应用的系统诊断与关于页面；四是 3 分钟项目介绍视频的录制剪辑。此外，四人各自撰写自己模块的实验报告内容。

为什么需要 Rust

- **文件系统操作**：遍历缓存目录、复制/打包用户数据、计算目录大小，这些系统级文件操作在 Rust 中高效且安全
- **ZIP 压缩/解压**：数据导出为 ZIP 文件、导入时解压恢复，压缩算法是 CPU 密集操作，Rust 性能优势明显
- **系统信息采集**：读 OS 版本、CPU/内存、磁盘空间等信息需要调用系统 API，JS 做不到
- **编译信息嵌入**：利用 Rust 的 `env!("CARGO_PKG_VERSION")` 和 `built` crate 在编译期注入版本信息，JS 无法获取

功能模块一：可视化设置面板

功能 D1：独立的设置窗口

效果描述：通过系统托盘右键菜单点击"设置"，弹出一个独立的设置窗口。

窗口外观：

- 标题栏显示"Pake Plus 设置"
- 窗口大小约 500×600 像素，居中显示，不可最大化/最小化
- 左侧为功能分类导航（广告拦截 / 离线缓存 / 剪贴板 / 数据 / 关于），右侧为对应的设置内容
- 整体设计风格简洁，深色顶栏 + 白色内容区

功能 D2：设置项详情

广告拦截设置区域：

- 总开关：广告拦截功能的启用/关闭（开关按钮，绿色=开，灰色=关）
- 规则显示：当前生效的拦截规则列表，可逐条查看
- 添加规则：输入框 + 添加按钮，支持添加自定义规则
- 删除规则：每条规则右侧有删除按钮（叉号）
- 规则统计：底部显示"已加载 XXX 条规则"

离线缓存设置区域：

- 功能开关：离线缓存的启用/关闭
- 缓存大小上限：滑块（50MB ~ 1GB），当前占用 / 上限用量显示
- 已缓存统计："1,247 个文件，占用 168 MB"
- 清除缓存按钮：红色按钮，点击弹出确认框
- 缓存命中率显示：如 "最近一小时命中率: 73%"

剪贴板管理设置区域：

- 功能开关：剪贴板监听的启用/关闭
- 最大记录数：下拉选择（500 / 1000 / 2000 / 5000）
- 保留天数：下拉选择（7 天 / 14 天 / 30 天 / 永久）
- 忽略短文本：开关，小于 2 字符的不记录
- 当前历史条数：显示统计 + "清除全部历史"按钮

数据管理设置区域：

- 当前数据占用空间：显示各模块数据大小（缓存 168 MB、剪贴板 2.3 MB、设置 2 KB）
- "导出所有数据"按钮：点击后选择保存路径，将全部数据打包为 `.pake-data.zip`
- "导入数据"按钮：选择 `.pake-data.zip` 文件，解压恢复到本地

关于/诊断区域：

- 应用版本：显示当前 Pake Plus 版本号（编译期获取）
- Rust 编译信息：编译器版本、target triple、编译时间
- 系统信息：OS 名称/版本、CPU 核心数、总内存/可用内存、磁盘剩余空间
- "复制诊断信息"按钮：生成报告文本写入剪贴板，方便反馈问题时使用

功能 D3：设置持久化

效果描述：用户在设置面板做的任何修改，点击"保存"按钮后立即写入本地 JSON 配置文件。下次启动应用时自动加载这些配置，无需重新设置。

持久化行为：

- 保存时即时写入磁盘，不依赖应用正常退出
- 文件存储在应用数据目录下（如 `C:\Users\xxx\AppData\Local\com.pake.myapp\pake-settings.json`）
- 文件格式为可读的 JSON，用户也可以手动编辑
- 首次启动时配置文件不存在，自动生成含默认值的配置文件
- 配置损坏时（JSON 格式错误），自动回退到默认值并重新生成

功能模块二：数据导出/导入

功能 D4：一键导出

效果描述：用户在设置面板的"数据"标签页点击"导出所有数据"，弹出系统文件夹选择对话框。用户选择保存路径后，系统遍历所有本地数据——缓存文件、剪贴板历史数据库、设置 JSON——打包为一个 `.pake-data.zip` 文件。

导出内容：

- 广告拦截的自定义规则配置
- 离线缓存的索引文件和部分常用缓存（按 LRU 取最新的 50MB，避免文件过大）
- 剪贴板历史记录数据库
- 全局设置 JSON

用户感知：

- 点击"导出" → 选择保存路径 → 进度条显示打包进度 → 完成提示"数据已导出至 C:\Users\xxx\Desktop\pake-data-20260712.zip"
- ZIP 文件命名含日期: `pake-data-YYYYMMDD.zip`
- 导出过程中应用可以正常使用

Rust 核心工作: `zip crate` 创建压缩文件、`walkdir` 遍历数据目录、`std::fs` 复制文件到临时目录、`serde_json` 序列化配置摘要到 `manifest.json` 写入 ZIP 作为索引。

功能 D5：一键导入

效果描述: 用户换电脑或重装系统后，点击"导入数据"选择之前的 ZIP 文件。系统解析 ZIP 内容，将各模块数据恢复到对应目录，覆盖或合并现有数据。

导入流程:

1. 选择 `.pake-data.zip` 文件
2. 系统读取 ZIP 中的 `manifest.json`，确认文件格式兼容
3. 弹出确认框，显示导入内容摘要（如"将恢复 1,247 个缓存文件、892 条剪贴板历史、7 条自定义规则"）
4. 用户确认 → 解压恢复 → 提示"数据已导入，请重启应用生效"

容错处理:

- ZIP 损坏 → 提示"文件格式不正确"
- 版本不兼容 → 提示"该数据来自旧版本，无法导入"
- 磁盘空间不足 → 提示"磁盘空间不足，需要 XX MB"

Rust 核心工作: `zip crate` 解压、反向遍历 `manifest` 恢复文件到目标路径、数据完整性校验（文件数量/大小比对）。

功能 D6：CLI 选项 --data-export

效果描述: 在打包时控制是否启用数据导出/导入功能。默认启用。

```
1 | pake https://example.com --data-export false
```

功能模块三：应用诊断与关于页

功能 D7：系统信息采集

效果描述: 设置面板的"关于"标签页显示应用和系统的诊断信息。Rust 后端在应用启动时采集系统信息，前端页面加载时获取并展示。

显示内容:

分类	信息项
应用信息	Pake Plus 版本号、Git commit hash、编译时间、target triple（如 <code>x86_64-pc-windows-msvc</code> ）
Rust 工具链	<code>rustc</code> 版本、 <code>cargo</code> 版本、编译 profile (release/debug)

分类	信息项
系统信息	OS 名称与版本、CPU 型号与核心数、总内存 / 可用内存、磁盘总空间 / 剩余空间
运行时状态	应用启动时间、运行时长、进程 ID
已启用功能	广告拦截 / 离线缓存 / 剪贴板管理 / 数据迁移（显示各功能的开启状态）

功能 D8：一键复制诊断报告

效果描述： 用户点击"复制诊断信息"按钮，系统将所有诊断信息格式化为纯文本，写入剪贴板。用户在提交 bug 或求助时可直接粘贴。

报告格式示例：

```
1 | Pake Plus v1.0.0 (commit: a1b2c3d, built: 2026-07-16)
2 | rustc 1.93.0, target: x86_64-pc-windows-msvc
3 | windows 11 Home China 24H2, CPU: 12 cores, RAM: 8.0/16.0 GB
4 | Disk: C: 45.2/256.0 GB free
5 | Enabled: adblock, cache, clipboard, data-export
6 | Uptime: 2h 34m
```

工作方式： 利用 Rust 的 `sysinfo` crate 采集系统信息，利用 `built` crate（编译期生成 `built.rs`）注入构建信息，`chrono` crate 格式化时间。在 JS 侧点击按钮 → IPC 调用 Rust 命令 → Rust 组装文本 → 写入剪贴板。

功能模块四：视频制作（3 分钟）

视频结构：

时间段	画面内容	画外音
0:00-0:15	标题画面：项目名称 + 团队信息	"Pake Plus，基于开源项目 Pake 的网页桌面应用增强工具"
0:15-0:40	演示原 Pake 打包一个含广告的网页，广告正常显示；切换 Pake Plus，同一网页广告被拦截，展示拦截计数	"原 Pake 没有任何广告过滤能力。我们新增了基于 EasyList 的广告拦截引擎"
0:40-1:05	正常浏览网页 → 断网 → 页面仍能正常打开（从缓存加载） → 展示缓存统计面板命中率	"离线缓存代理在 HTTP 层自动缓存资源，断网也能访问已浏览页面"
1:05-1:35	Ctrl+Shift+V 打开剪贴板面板 → 搜索关键词过滤历史 → 点击一条记录复制 → 粘贴到编辑器	"系统级剪贴板管理，自动记录、全文搜索、一键复用"
1:35-1:50	右键托盘 → 设置面板 → 修改各项配置 → 切换到数据页 → 一键导出 ZIP	"可视化设置面板管理所有配置，还支持数据一键导出迁移"
1:50-2:05	切换到关于页 → 查看系统诊断信息 → 复制诊断报告	"内置系统诊断功能，可一键复制报告用于问题反馈"

时间段	画面内容	画外音
2:00-2:25	对比表格展示：Pake vs Pake Plus 的功能差异	"相比原 Pake，我们新增了 4 大功能模块、20+ 项子功能"
2:25-2:50	四位成员的模块分工介绍（每人照片/名字 + 负责模块）	"项目由四位同学分工协作完成"
2:50-3:00	结束画面：GitHub 链接 + 致谢	"感谢观看"

功能模块五：实验报告（每人各自撰写）

四人各自撰写自己模块的实验报告内容（约 800-1200 字/人），包括：模块设计思路、核心算法或流程说明、实现效果截图、遇到的问题与解决方案。最终由小组共同评审，统一整合为一个 PDF 提交。

报告章节与撰写分工：

章节	内容
1. 项目名称与团队介绍	项目简介、成员学号与分工
2. 项目背景与目标	选题原因、Pake 原项目简介、改进目标
3. 系统设计与整体架构	整体架构图、各模块关系、技术栈
4.1 广告拦截模块	设计思路、规则引擎原理、效果截图
4.2 离线缓存代理模块	设计思路、HTTP 拦截代理、LRU 淘汰算法、效果截图
4.3 剪贴板管理模块	设计思路、平台 FFI 剪贴板监听、历史存储方案、效果截图
4.4 设置面板与数据迁移与诊断	设计思路、配置管理、数据打包、诊断信息采集、效果截图
5. 实验结果	各模块功能测试汇总、命令行示例
6. 开源项目参考说明	Pake 项目信息、引用部分、与原项目对比
7. 总结与展望	项目总结、不足与改进方向

每人将自己负责的章节写成独立文档，最后小组统一整合排版、处理交叉引用，编译为 PDF 提交。

六、人员分工汇总

成员	负责模块	功能数	工作内容
成员 A	广告/跟踪拦截	6 项	EasyList 规则引擎、网络请求拦截、DOM 元素隐藏、自定义规则、拦截统计、CLI 选项
成员 B	离线缓存代理	7 项	HTTP 代理拦截、磁盘缓存管理、LRU 淘汰算法、离线模式、缓存统计面板、CLI 选项

成员	负责模块	功能数	工作内容
成员 C	剪贴板管理	9 项	系统剪贴板监听 (FFI) 、历史记录面板、全文搜索、一键复用、隐私过滤、自动清理、快捷键入口、CLI 选项
成员 D	设置面板 + 数据迁移 + 诊断 + 视频	8 项	设置窗口 UI、数据导出/导入 (ZIP 打包解压) 、系统诊断信息采集 (sysinfo) 、关于页面、3 分钟视频录制

各模块独立性说明

四个模块在功能层面完全独立，互不依赖：

- A 的广告拦截不需要 B/C/D 的任何功能即可运行
- B 的离线缓存不需要 A/C/D 的任何功能即可运行
- C 的剪贴板管理不需要 A/B/D 的任何功能即可运行
- D 的设置面板是一个独立窗口，通过读取/写入配置文件来管理 A/B/C 模块的设置，但不影响各模块独立运行

因此四人可以完全并行开发，最后通过 D 的设置面板和共享配置文件进行统一集成。
